CAR KART

Cloud-Based Car Ordering Chatbot
Using Amazon Lex V2 and AWS Lambda

BASIC PROJECT REPORT

Submitted by

Student Name: _____

Course / Department: _____

College / Institute: _____

Academic Year: _____

Technologies: Amazon Lex V2 • AWS Lambda • Python • Boto3 • AWS IAM

This report documents the design, implementation, testing and result of the CarKart conversational car-ordering project.

TABLE OF CONTENTS

Chapter 1 — Introduction

Chapter 2 — Problem Statement

Chapter 3 — Objectives

Chapter 4 — Technologies Used

Chapter 5 — System Architecture

Chapter 6 — Implementation

Chapter 7 — Lambda Order Processing

Chapter 8 — Testing

Chapter 9 — Results

Chapter 10 — Advantages

Chapter 11 — Limitations

Chapter 12 — Future Scope

Chapter 13 — Conclusion

References —

Note: This is a basic project report. Screenshot areas are intentionally marked so the exact screenshots from the completed AWS Lex, Lambda and local Python practical can be inserted before submission.

CHAPTER 1 — INTRODUCTION

1.1 What is CarKart?

CarKart is a cloud-based conversational chatbot designed to demonstrate how a user can place a car-order request through a natural-language conversation. The chatbot is built using Amazon Lex V2 for conversation management and AWS Lambda for serverless order processing.

The chatbot collects customer information such as name, email, contact number and delivery address. It then collects the selected car model, colour and preferred delivery date. Before processing the request, CarKart asks the user to confirm the order. After confirmation, the Lambda function generates an order ID, looks up an assumed project price for the selected car and returns a formatted order summary.

The project also includes a local Python client using Boto3. This demonstrates that the Lex bot can be accessed outside the AWS console through a Python program.

1.2 What is Conversational AI?

Conversational Artificial Intelligence enables computer systems to communicate with users using natural language. Instead of requiring users to navigate multiple screens and forms, a conversational system can ask questions one at a time and collect the required information through a dialogue.

In CarKart, conversational AI is used to guide the customer through the car-ordering process. Amazon Lex provides natural-language understanding, intent recognition and slot-based information collection.

1.3 Why Amazon Lex?

Amazon Lex V2 is suitable for this project because it provides a managed service for building conversational interfaces. It supports intents, sample utterances, slots, prompts, confirmation and integration with AWS Lambda. This allows the project to demonstrate both conversational AI and serverless cloud computing without building a complete natural-language processing system from scratch.

CHAPTER 2 — PROBLEM STATEMENT

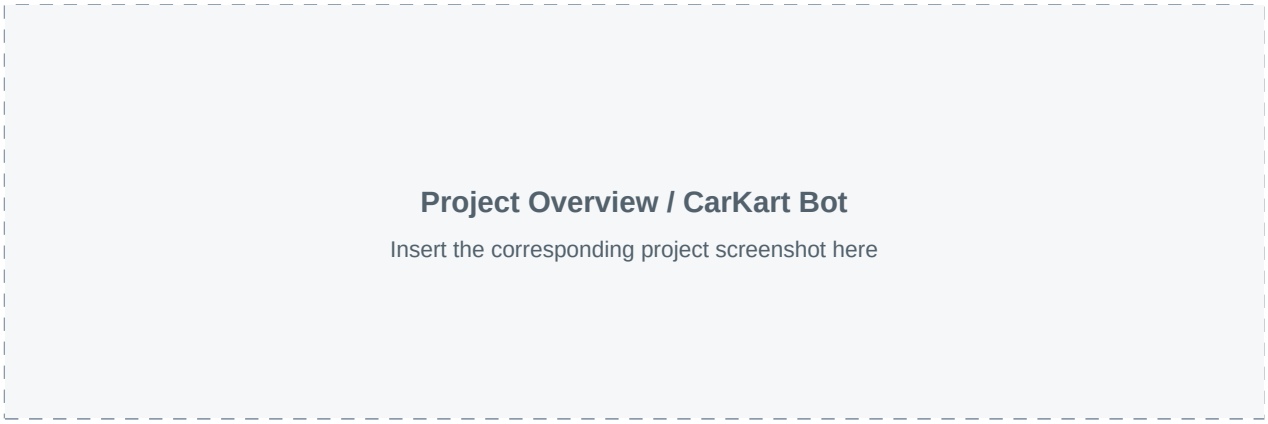
Traditional online car-ordering interfaces normally require users to navigate forms, menus and multiple pages. For a simple academic demonstration, there is a need for an easier interaction model in which a user can provide information conversationally.

The problem addressed by CarKart is therefore to design a simple conversational system that can collect customer and car information, validate the conversation flow, confirm the order and automatically process the request using a cloud-based backend.

The system should be simple enough for a basic project while demonstrating practical cloud concepts such as managed AI services, serverless computing, IAM-based access and programmatic interaction with AWS.

Problem statement:

“Design and implement a cloud-based conversational chatbot that allows a customer to provide car-order details through Amazon Lex V2 and processes the confirmed order using AWS Lambda.”



Suggested screenshot: Amazon Lex CarKart bot overview

CHAPTER 3 — OBJECTIVES

The main objectives of the CarKart project are:

- To develop a conversational car-ordering chatbot using Amazon Lex V2.
- To recognize a user's intention to purchase or order a car.
- To collect customer details using Lex slots.
- To provide a predefined list of car models and colours.
- To collect a preferred delivery date.
- To confirm the order before fulfillment.
- To use AWS Lambda for serverless order processing.
- To perform price lookup using a predefined Python dictionary.
- To generate a unique order ID for every confirmed order.
- To format the delivery date and Indian currency for a readable result.
- To test the chatbot through both the AWS Lex test console and a local Python/Boto3 application.

Project Scope

The scope is intentionally limited to a basic working prototype. Prices are assumed project values and are not connected to a real dealership or live vehicle inventory. No payment is collected and no real order is created with a car manufacturer.

CHAPTER 4 — TECHNOLOGIES USED

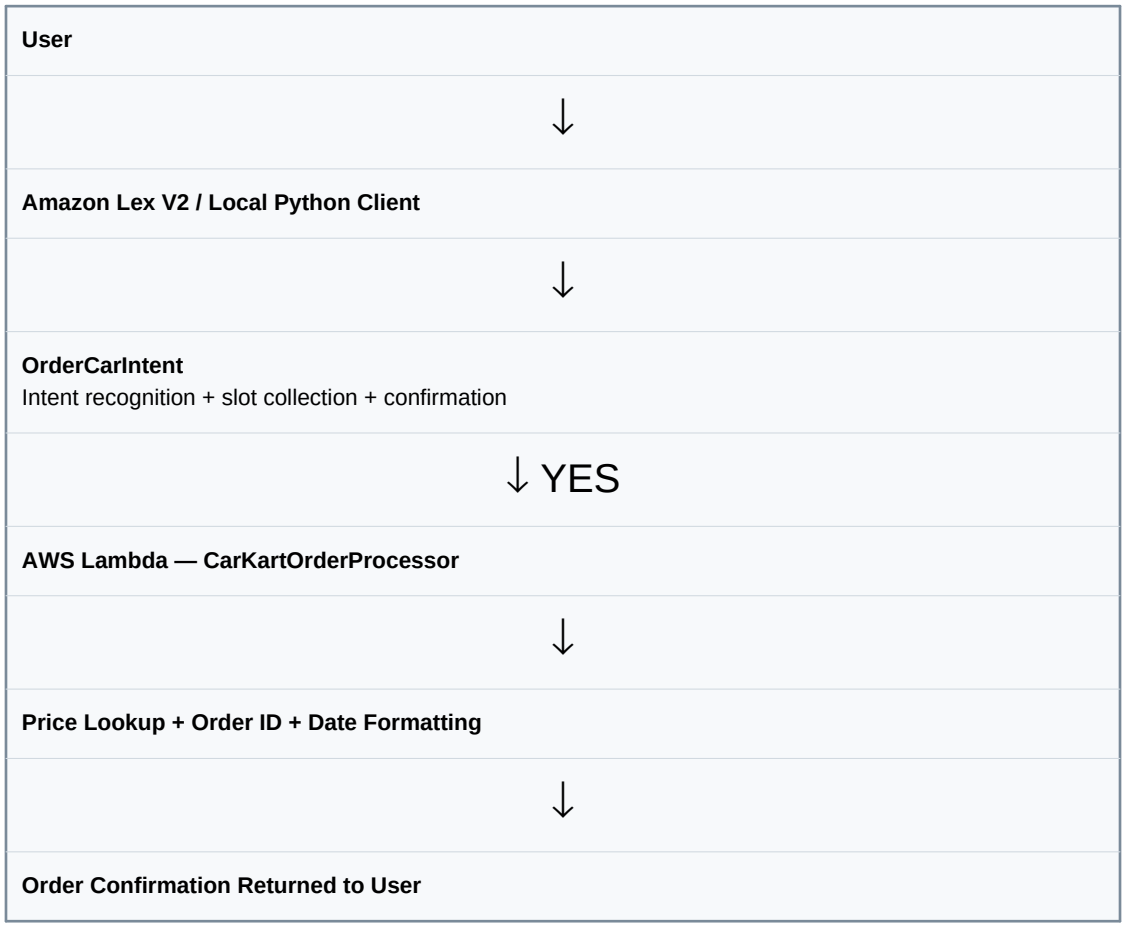
Technology	Purpose in CarKart
Amazon Lex V2	Builds the conversational chatbot, intents, slots, prompts and confirmation flow.
AWS Lambda	Processes the confirmed order and returns the final order summary.
Python	Used for Lambda implementation and the local chatbot client.
Boto3	AWS SDK for Python used by the local application to communicate with Amazon Lex V2.
AWS IAM	Provides controlled AWS access and permissions for the project.
AWS Cloud	Hosts the managed chatbot and serverless processing components.
Windows / Command Prompt	Used as the local environment for Python and Boto3 testing.

AWS Components Used

- Lex bot: **CarKart**
- Locale: **English (US) — en_US**
- Intent: **OrderCarIntent**
- Alias: **TestBotAlias**
- Lambda function: **CarKartOrderProcessor**
- Custom slot types: **CarModelType** and **CarColourType**

CHAPTER 5 — SYSTEM ARCHITECTURE

5.1 Architecture Overview



5.2 Architecture Explanation

- **User layer:** The customer starts the conversation by expressing a car-order request.
- **Conversation layer:** Amazon Lex V2 identifies OrderCarIntent and asks for missing slot values.
- **Slot collection:** CustomerName, Email, ContactNumber, Address, CarModel, CarColour and DeliveryDate are collected.
- **Confirmation:** Lex asks the user to confirm the complete order before fulfillment.
- **Processing layer:** AWS Lambda receives the confirmed request and performs the business logic.
- **Response layer:** Lambda returns an order ID, assumed price, formatted date and customer/car details.
- **Local access:** A Python/Boto3 client can communicate with the Lex bot outside the AWS console.

Figure 5.1 — CarKart System Architecture

The architecture follows a simple serverless pattern. No dedicated server is required for the order-processing logic.

CHAPTER 6 — IMPLEMENTATION

6.1 Creating the Amazon Lex V2 Bot

A Lex V2 bot named CarKart was created in the us-east-1 region with the English (US) locale. The bot is configured for a car-ordering conversation.

Figure 6.1 — CarKart Lex Bot

Insert the corresponding project screenshot here

Insert screenshot of CarKart bot / locale configuration

6.2 Creating CarModelType

A custom slot type named CarModelType was created to restrict the car selection to a predefined set of 25 models. Synonyms were also configured for commonly used short names such as Creta, Thar, Nexon and Swift.

Figure 6.2 — CarModelType

Insert the corresponding project screenshot here

Insert screenshot showing the custom car model slot type

The project model list includes Hyundai Creta, Hyundai Venue, Hyundai Verna, Hyundai i20, Hyundai Alcazar, Tata Nexon, Tata Punch, Tata Harrier, Tata Safari, Tata Altroz, Mahindra Thar, Mahindra XUV700, Mahindra Scorpio, Mahindra Scorpio N, Mahindra Bolero, Toyota Fortuner, Toyota Innova Crysta, Toyota Hyryder, Kia Seltos, Kia Sonet, Kia Carens, Maruti Swift, Maruti Baleno, Maruti Brezza and Honda City.

6.3 Creating CarColourType

A custom slot type named CarColourType was created with five values: White, Black, Red, Blue and Silver.

Figure 6.3 — CarColourType

Insert the corresponding project screenshot here

Insert screenshot of the colour slot type

6.4 Creating OrderCarIntent

The main intent is OrderCarIntent. Sample utterances include “I want to buy a car”, “I want to purchase a car”, “I want to order a car”, “I want a new car” and similar expressions.

Figure 6.4 — OrderCarIntent utterances

Insert the corresponding project screenshot here

Insert screenshot of intent sample utterances

6.5 Configuring Slots

The intent uses the following required slots: CustomerName, Email, ContactNumber, Address, CarModel, CarColour and DeliveryDate. Free-form input is used for customer information while custom slot types are used for car model and colour. DeliveryDate uses AMAZON.Date.

Figure 6.5 — Customer and car slots

Insert the corresponding project screenshot here

Insert screenshot showing configured slots and prompts

6.6 Intent Confirmation

After all required information is collected, Lex asks the user to confirm the order. If the user answers yes, the intent proceeds to fulfillment. If the user declines, the order is cancelled.

Figure 6.6 — Confirmation configuration

Insert the corresponding project screenshot here

Insert screenshot of the Lex intent confirmation settings

6.7 Lambda Fulfillment Configuration

The fulfillment step is connected to the Lambda function CarKartOrderProcessor. Dialog Code Hook is not required for this basic implementation; Lambda is used for fulfillment after confirmation.

Figure 6.7 — Lambda association / fulfillment

Insert the corresponding project screenshot here

Insert screenshot showing TestBotAlias and Lambda association

6.8 Building the Bot

After configuring the intent, slots, confirmation and fulfillment, the bot was built successfully. A successful build confirms that the bot configuration is valid and ready for testing.

Figure 6.8 — Successful bot build

Insert the corresponding project screenshot here

Insert screenshot showing successful build status

CHAPTER 7 — LAMBDA ORDER PROCESSING

7.1 Role of AWS Lambda

AWS Lambda provides the backend business logic for CarKart. The function receives the values collected by Lex and creates the final order response. Because Lambda is serverless, there is no need to manage a continuously running application server.

7.2 Price Lookup

The project uses a predefined Python dictionary containing assumed prices for the supported car models. These values are used only for project demonstration and do not represent live market prices.

```
CAR_PRICES = {
"hyundai creta": 1500000,
"mahindra thar": 1800000,
"tata nexon": 1100000,
"toyota fortuner": 3500000,
"maruti swift": 800000,
# other supported models...
}
```

7.3 Model Alias Handling

The Lambda function also supports short model names by mapping them to the corresponding full model name. For example, “Thar” is mapped to “Mahindra Thar”.

7.4 Order ID Generation

After confirmation, a unique-looking project order identifier is generated using the current date and a four-digit number. Example: CK-20261007-8699.

```
order_id = f"CK-{date.today().strftime('%Y%m%d')}-{random.randint(1000,9999)}"
```

7.5 Date Formatting

Lex can provide the delivery date in ISO-style form such as 2027-09-07. Lambda converts this into a user-friendly format such as 07 September 2027.

7.6 Indian Currency Formatting

The price is formatted using the Indian numbering system. For example, an assumed value of 1800000 is displayed as ■18,00,000.

7.7 Order Confirmation

The final Lambda response contains the order ID, customer details, car model, colour, price and delivery date. The response is returned to Lex using a Close/Fulfilled response so that the conversation ends with the completed order summary.

Figure 7.1 — Lambda code

Insert the corresponding project screenshot here

Insert screenshot of the final CarKartOrderProcessor code

CHAPTER 8 — TESTING

8.1 Testing Approach

Testing was performed at two levels: first through the Amazon Lex test console and then through a local Python application using Boto3. The objective was to verify the complete conversation, slot collection, confirmation, Lambda invocation and final output.

Test Case	Input / Action	Expected Result	Status
TC-01	Start with “I want to buy a car”	OrderCarIntent starts	Pass
TC-02	Provide customer details	Required slots collected	Pass
TC-03	Provide a supported car	CarModel accepted	Pass
TC-04	Provide a supported colour	CarColour accepted	Pass
TC-05	Provide delivery date	Date accepted	Pass
TC-06	Confirm with yes	Lambda fulfillment runs	Pass
TC-07	Confirm with no	Order cancelled	Pass
TC-08	Run local Python/Boto3 client	Same Lex conversation works	Pass

8.2 AWS Lex Test

The AWS Lex test console was used to complete a full order. The conversation successfully collected the required information, asked for confirmation, invoked Lambda and displayed the generated order summary.



Insert screenshot of the successful end-to-end Lex order

8.3 Cancelled Order Test

A negative-path test was included by selecting the confirmation response “No”. The expected behavior is that Lex does not invoke the successful fulfillment path and instead displays the configured cancellation message.

Figure 8.2 — Cancelled order

Insert the corresponding project screenshot here

Insert screenshot of the conversation where the user declines confirmation

8.4 Different Car Test

A different car model was tested to verify that the price lookup and final response change according to the selected model. For example, Mahindra Thar uses the assumed project price of ■18,00,000.

Figure 8.3 — Different car test

Insert the corresponding project screenshot here

Insert screenshot showing a different car model and its result

8.5 Local Python / Boto3 Test

The local application was executed from Windows Command Prompt. The Python program communicates with Amazon Lex V2 using Boto3 and prints the conversation prompts and final response in the terminal.

Figure 8.4 — Local Python test

Insert the corresponding project screenshot here

Insert the successful CarKart_Local.py terminal screenshot

CHAPTER 9 — RESULTS

The CarKart prototype successfully completed the intended conversational workflow. The user can begin with a natural-language request, provide the required details, confirm the order and receive a generated order summary.

A successful local test produced the following representative result:

```
CarKart: Welcome to CarKart! May I know your name?
CarKart: Thank you. Please provide your email address.
CarKart: Please provide your 10-digit contact number.
CarKart: Please provide your complete delivery address.
CarKart: Great! What is your favourite car?
CarKart: Which colour would you like for your car?
CarKart: What date would you like your car to be delivered?
CarKart: Please confirm that you would like to place this car order.

■ CarKart Order Confirmed!

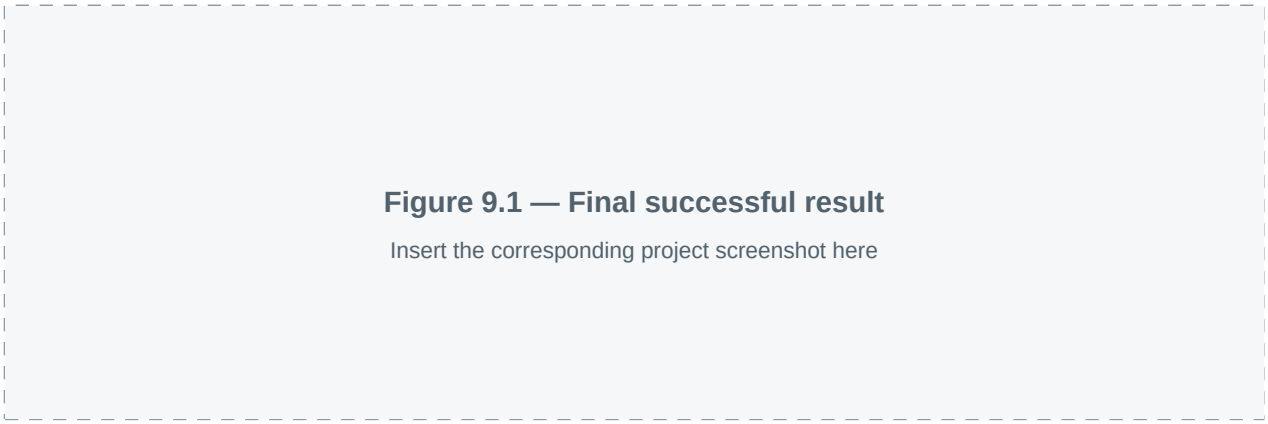
Order ID: CK-20261007-8699

Customer Details
Name: pratima
Email: paratima@mail
Contact Number: 9145342654
Delivery Address: mumbai

Car Details
Car Model: Mahindra Thar
Colour: White
Car Price: ■18,00,000
Delivery Date: 07 September 2027

Thank you for choosing CarKart! ■
```

The displayed values above are a demonstration result from the project. Prices are assumed values for academic use and are not live vehicle prices.



Insert the strongest final-output screenshot here

CHAPTER 10 — ADVANTAGES

- Conversational and easy-to-use interaction.
- Uses managed AWS services, reducing infrastructure management.
- Serverless Lambda processing can scale automatically.
- Slot-based design ensures required order information is collected.
- Intent confirmation reduces accidental order processing.
- Can be tested both from AWS Lex and a local Python application.
- The architecture is simple and suitable for learning cloud and conversational-AI concepts.
- The project can be extended with databases, APIs and external channels.

Practical Learning Benefits

The project demonstrates several real-world cloud concepts together: managed AI services, serverless computing, IAM, Python SDK usage, event-driven processing and basic business logic. This makes it suitable as a compact cloud-computing academic project.

CHAPTER 11 — LIMITATIONS

- Car prices are predefined assumptions and are not fetched from a live pricing system.
- The prototype does not store confirmed orders in a database.
- No real payment gateway is connected.
- No vehicle inventory or dealership availability is checked.
- The order ID is generated for demonstration and is not a persistent transactional identifier.
- Customer authentication and account management are not implemented.
- The current interface is primarily the Lex test console or local Python client.
- Error handling and production-grade validation are intentionally kept simple.

These limitations are acceptable for a basic academic prototype. They also identify clear areas for future development.

CHAPTER 12 — FUTURE SCOPE

The basic CarKart prototype can be extended into a more complete cloud application by adding the following capabilities:

Future Enhancement	Possible Implementation
Database integration	Store customer and order records using Amazon DynamoDB.
Real-time inventory	Connect to a vehicle inventory service through an API.
Payment gateway	Add a secure payment service after order confirmation.
Dealer integration	Send confirmed orders to a dealer or sales-management system.
WhatsApp integration	Expose the conversational experience through a supported messaging channel.
Authentication	Use a secure identity service for customer accounts.
Real-time pricing	Retrieve current model/variant prices through an API.
Amazon API Gateway	Expose backend APIs and integrate additional services.
Order tracking	Provide order status, delivery updates and notifications.
Notifications	Add email/SMS notifications for order confirmation and delivery.
Analytics	Track conversations, popular models and order trends using suitable AWS analytics services.

A production version could combine Lex, Lambda, API Gateway, DynamoDB, authentication and notification services to create a complete serverless ordering platform.

CHAPTER 13 — CONCLUSION

CarKart demonstrates how cloud services can be combined to build a functional conversational application. Amazon Lex V2 manages the natural-language conversation, collects required information through slots and performs intent confirmation. AWS Lambda provides the backend processing required to calculate an assumed price, generate an order ID, format the delivery date and return the completed order summary.

The project was successfully tested through the AWS Lex test environment and through a local Python application using Boto3. The successful end-to-end flow shows that the chatbot can collect customer and car details, confirm the request and produce a readable order result.

Although the current implementation is intentionally basic, its serverless architecture provides a strong foundation for future enhancements such as DynamoDB storage, real-time inventory, payment processing, authentication, dealer integration and order tracking.

Overall, CarKart meets the objectives of a basic cloud-computing project by demonstrating conversational AI, serverless processing and practical AWS integration in a single application.

REFERENCES

1. Amazon Web Services — Amazon Lex V2 Documentation.

2. Amazon Web Services — AWS Lambda Documentation.

3. Amazon Web Services — Boto3 Documentation / AWS SDK for Python.

4. Amazon Web Services — AWS Identity and Access Management (IAM) Documentation.

5. Python Documentation — Python language and standard library reference.

Submission Checklist

- Replace the cover-page placeholders with your name, college, course/department and academic year.
- Insert the actual screenshots into the marked screenshot areas.
- Keep the strongest AWS Lex, Lambda, successful build, AWS test and local Python screenshots.
- Review the final PDF once after inserting screenshots and update the table of contents/page numbers if your college requires them.

End of Report